

Table of Contents

Studium — Data Layer Specification

Subsystem 1 of 7. Version 1.1. Status: built and ratified.

Version 1.0 was built against by a solo developer and reviewed against itself and the existing draft. Twenty-nine defects were found; five were blocking (the schema could not have been created as written), the remainder were internal contradictions or design risks. Version 1.1 folds every ratified correction into a single authoritative document, so no future subsystem is written against a spec the code no longer matches. The full history of changes is recorded in §17.

1. Overview

This document specifies the persistence layer for Studium: every table, column, constraint, index, and access pattern the system depends on. Six subsystem specs will follow (agent runtime, retrieval, frontend, ingestion, evaluation, infrastructure) and every one of them will reference schemas defined here. Getting this right first is the argument made in the planning phase and this document is the payoff.

The reader is assumed to be a senior engineer comfortable with Postgres, familiar with LLM application architecture, and able to fill in the mechanical details of migration files, connection pooling, and ORM setup without further instruction. Where this document is explicit it is because the decision is load-bearing; where it is silent it is because the standard practice is fine.

2. Scope and non-goals

In scope.

- The full Postgres schema for all nine modules of Studium, including the modules deferred past MVP (concept graph as a user-facing surface, multi-voice library, review cycle, orientation, and summative assessment). The schema anticipates them so no migration is needed when they ship.
- Indexing strategy, foreign-key topology, check constraints, and query patterns for the common access paths of every agent in the system.
- Data lifecycle rules: retention windows, deletion semantics, PIPEDA-aligned right-to-erasure handling, backup and restore posture.
- Migration path from the existing draft's JSON-file persistence to the new Postgres model.

- The migration tooling and workflow (Alembic) for evolving the schema over the life of the product.

Explicitly out of scope.

- Vector store details beyond the pgvector column definitions. Chunking strategy, embedding model choice, hybrid search ranking, and reranker configuration live in the Retrieval spec (subsystem 3).
- Agent behavior. What the Tutor does with a `journal_entries` row belongs in the Agent Runtime spec (subsystem 2). This document specifies only that the row exists, what it holds, and how it is accessed.
- Prompt content. Agent prompts appear in the Agent Runtime spec.
- Deployment topology. Postgres sizing, connection pool configuration, backup targets, and multi-AZ posture are in Infrastructure (subsystem 7).
- Frontend concerns. The UI never talks to Postgres directly.

3. Design principles

The following invariants are held by the schema. Any subsequent spec that appears to violate one of them is either wrong or has surfaced a real problem this document should address; there is no third case.

Deterministic gating in application code, not model judgment. No table stores a value computed by asking a model “did the learner pass?” or “is this concept mastered?”. Mastery is a numerical quantity derived by the BKT update rule from `mastery_events`; passing is a comparison against a fixed threshold; unlocking a concept is a graph reachability test over `concept_edges`. This principle is inherited directly from the existing draft’s `progress.py`, where it is already right, and it applies everywhere gating decisions are made.

Agent context isolation. The schema does not carry a single “conversation” table shared across agents. Each agent’s turns are stored as `session_turns` rows tagged with the acting agent, so an Evaluator grading a response cannot accidentally read a Tutor’s encouraging preamble as evidence of correctness, and a Tutor asking a follow-up question is not conditioned on the Confusion-Tracker’s private hypothesis about the learner’s gap. Agents share the session, not the context; the schema enforces this by structure.

Mastery model separation from any single agent. The `concept_mastery` table is the system’s canonical statement of what the learner knows. It is written to by any agent whose interaction produces evidence, but no agent owns it. This is what allows the Reviewer, the Curator, and the frontend to all read a consistent picture without asking an LLM to synthesize one on demand.

Every mutation is auditable. The append-only tables (`mastery_events`, `journal_events`, `review_events`, `agent_traces`, `audit_log`) exist so that any current-state value can be reconstructed from its history. This costs storage. It buys the ability to debug a “why does the system think I know this?” question with a query, and to re-derive mastery under a corrected BKT parameterization without losing evidence.

Content is versioned, learners are not tied to versions. A lecture segment can be regenerated, a rubric criterion can be edited, a concept description can be refined. Prior generations are retired (`superseded_by`, `retired_at`) rather than deleted, so a learner's assessment attempt from three weeks ago can still be shown alongside the exact prompt they saw. New learners get the current version; old evidence is never invalidated by a content update.

JSONB for shape, columns for semantics. Structured data that the schema needs to constrain, index, or join on is a column with a real type. Structured data that is only ever read as a blob by one consumer is JSONB. The line is drawn deliberately in each table below; a rule of thumb: if a query would ever want to `WHERE` on a value, it is a column, not JSONB.

No premature multi-tenancy. Studium has three known users at MVP and possibly a classroom later. The schema does not carry `tenant_id` on every table. If institutional deployment happens, tenant separation will be handled by database-per-tenant, which is cleaner than row-level tenant filtering for the read patterns Studium has. This document notes where the boundary would fall if that day comes.

Cost tracking is first-class. Every LLM call writes to `agent_traces` with token counts and cost; a daily `cost_ledger` roll-up supports per-user budget enforcement without scanning traces at query time. This is spec'd because uncontrolled LLM cost is the mode-of-failure most likely to end the project in weeks 8–12 if left unmeasured.

4. Technology stack

Database. Postgres 16.x. Not 15, not 17-beta. Postgres 16 gives us `MERGE`, better logical replication, improved parallel query, and is what Fly.io currently ships as its default managed Postgres image.

Extensions.

- `pgvector` (0.7 or later) for embedding storage and cosine/L2 similarity search.
- `pg_trgm` for trigram indexes on titles and search-adjacent text columns.
- `pg_uuidv7` for `uuid_generate_v7()`, used for all primary keys (see conventions). If unavailable in the deploy target, a `plpgsql` fallback is provided in §6.0. Note: `uuid-oss` does *not* provide a v7 generator; v7 is a 2024 addition to the UUID spec and requires a dedicated extension or a hand-rolled function.
- `citext` for case-insensitive email storage.
- `pgcrypto` for `gen_random_bytes` and hash functions used in portfolio manifest signing.

Migration tooling. Alembic (Python-native, integrates with the FastAPI backend already in the draft). Migration files live in `backend/migrations/`, are numbered and dated, and are run automatically on application startup in development and via a deploy hook in staging and production. Downgrade migrations are required for every up-migration; the CI check that verifies this is specified in §14.

ORM. SQLAlchemy 2.x with the modern typed declarative style. The reasoning: the existing draft uses Pydantic for schema and no ORM (raw JSON files). Moving to Postgres wants an ORM for connection management, transaction scoping, and eager-loading control; SQLAlchemy is the mature choice and integrates with FastAPI cleanly through a session dependency. Pydantic models continue to be the wire format for the API; SQLAlchemy models are internal to the backend.

Connection pooling. PgBouncer in transaction mode, sized in the Infrastructure spec. This document notes that transaction-mode pooling forbids session-scoped features Studium does not use (SET, prepared statements without explicit lifecycle, LISTEN/NOTIFY); nothing in the query patterns below relies on any of these.

Timezone. All timestamps are TIMESTAMPTZ. Application code stores UTC. Client-side display timezone comes from the user's preferences.

5. Naming and type conventions

Primary keys. UUIDv7, generated by `uuid_generate_v7()`. UUIDv7 is time-ordered, so index locality is preserved without paying for a sequential integer and without exposing sequential IDs in URLs. Column name is always `id`, type `UUID`, default `uuid_generate_v7()`.

Foreign keys. Column name is `{referenced_table_singular}_id`. Every foreign key is declared with an explicit ON DELETE policy — usually CASCADE for owned data, RESTRICT for reference data, SET NULL for optional non-owning references. Every foreign key has an index unless the reference is unique (which creates an index anyway).

Timestamps. `created_at` and `updated_at` on every mutable table, both TIMESTAMPTZ NOT NULL DEFAULT NOW(). `updated_at` is maintained by a trigger installed once (see §6.0). Append-only event tables carry only `created_at`.

Enums. Postgres native ENUM types for closed sets used in three or more tables (e.g., agent identity), TEXT columns with CHECK constraints for closed sets used in one or two tables. This avoids the migration pain of Postgres enum evolution for values that are unlikely to change often.

JSONB. Column type is JSONB, not JSON. Default is `'{}'::jsonb` for object-shaped columns and `'[]'::jsonb` for array-shaped. GIN indexes are added only where a query patterns needs to search into the JSON; §7 lists them.

Booleans. NOT NULL with an explicit default, always. A nullable boolean is nearly always a modeling mistake.

Strings. TEXT for everything. No VARCHAR(n). Length constraints live in application code or CHECK constraints where they matter.

Money. NUMERIC(10, 4) for USD amounts (four decimal places accommodates fractional-cent per-token pricing without floating-point drift). Column names end in `_usd`.

Deletion. Soft delete via `deleted_at` `TIMESTAMPTZ` on tables where recovery matters (users, sources, subjects, content_artifacts). Hard delete elsewhere. A partial index `WHERE deleted_at IS NULL` supports the common “active only” queries without a query rewrite.

6. Schema reference

6.0. Common infrastructure

Applied once, before any table definitions.

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "citext";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
CREATE EXTENSION IF NOT EXISTS "vector";

-- Preferred: install the pg_uuidv7 extension for a native, tested v7
-- generator.
-- CREATE EXTENSION IF NOT EXISTS "pg_uuidv7";
-- Reference: https://github.com/fboulnois/pg\_uuidv7
--
-- Fallback: if pg_uuidv7 is unavailable on the deploy target (some managed
-- Postgres offerings do not permit arbitrary extensions), the plpgsql
-- function
-- below produces a compliant UUIDv7 value. It must produce exactly 32 hex
-- characters – the v1.0 draft of this function returned 33 and would have
-- failed every insert on the ::uuid cast. This version is component-by-
-- component to make the character count auditable.
--
-- UUIDv7 layout (128 bits total):
-- 48 bits unix_ts_ms (12 hex chars)
-- 4 bits version=7 (1 hex char, literal '7')
-- 12 bits rand_a (3 hex chars)
-- 2 bits variant=10 } combined into 1 byte
-- 6 bits rand_b_high } (2 hex chars, always 0x80-0xbf)
-- 56 bits rand_b_low (14 hex chars)
--
-- -----
-- 32 hex chars
CREATE OR REPLACE FUNCTION uuid_generate_v7() RETURNS uuid AS $$
DECLARE
    v_time_ms bigint;
    v_rand bytea;
    v_rand_a int;
    v_variant_byte int;
    v_hex text;
BEGIN
    v_time_ms := (extract(epoch from clock_timestamp()) * 1000)::bigint;
    v_rand := gen_random_bytes(10);

    -- 12 bits of rand_a assembled from the low nibble of byte 0 and all of
```

```

byte 1
v_rand_a := ((get_byte(v_rand, 0) & 15) << 8) | get_byte(v_rand, 1);

-- Byte 2: force top two bits to '10' (RFC 4122 variant), keep low 6 as
random
v_variant_byte := (get_byte(v_rand, 2) & 63) | 128;

v_hex :=
    lpad(to_hex(v_time_ms), 12, '0')           -- 12 chars
    || '7'                                     -- 1 char
    || lpad(to_hex(v_rand_a), 3, '0')          -- 3 chars
    || lpad(to_hex(v_variant_byte), 2, '0')    -- 2 chars
    || encode(substring(v_rand FROM 4 FOR 7), 'hex'); -- 14 chars

RETURN v_hex::uuid;
END;
$$ LANGUAGE plpgsql VOLATILE;

-- The fallback must be exercised against a live Postgres instance before
-- being trusted in production. A runtime test lives in
-- backend/tests/schema/test_uuid_generator.py.

-- updated_at maintenance
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Applied to each mutable table individually:
-- CREATE TRIGGER trg_<table>_updated_at BEFORE UPDATE ON <table>
-- FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

Shared enum types used across multiple tables:

```

CREATE TYPE agent_identity AS ENUM (
    'learner', 'orchestrator', 'curator', 'lecturer', 'tutor',
    'evaluator', 'confusion_tracker', 'reviewer', 'system'
);

CREATE TYPE session_mode AS ENUM (
    'orientation', 'lecture', 'tutorial', 'lab', 'review',
    'office_hours', 'summative_assessment'
);

CREATE TYPE artifact_status AS ENUM (
    'draft', 'reviewed', 'active', 'retired'
);

```

```
CREATE TYPE license_kind AS ENUM (
  'public_domain', 'cc_by', 'cc_by_sa', 'cc_by_nc',
  'user_uploaded', 'permission_granted', 'fair_use'
);
```

6.1. Identity and users

Four tables: users, learner profile, auth sessions, and a preferences blob for interface and pedagogical settings.

users

```
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  email       CITEXT UNIQUE NOT NULL,
  display_name TEXT NOT NULL,
  password_hash TEXT,           -- NULL if using OAuth only
  auth_provider TEXT NOT NULL DEFAULT 'local'
    CHECK (auth_provider IN ('local', 'google', 'github')),
  role        TEXT NOT NULL DEFAULT 'learner'
    CHECK (role IN ('learner', 'reviewer', 'admin')),
  locale      TEXT NOT NULL DEFAULT 'en-CA',
  timezone    TEXT NOT NULL DEFAULT 'America/Toronto',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at  TIMESTAMPTZ
);

CREATE INDEX idx_users_email_active ON users (email) WHERE deleted_at IS NULL;
CREATE TRIGGER trg_users_updated_at BEFORE UPDATE ON users
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

The role column exists to distinguish the human-in-the-loop reviewer role (you, during MVP) from ordinary learners. It gates access to the review queue and admin endpoints. admin is reserved and unused at MVP.

user_profiles

One-to-one with users. Split from users because profile data is larger, mutates more frequently, and is not needed on every auth check.

```
CREATE TABLE user_profiles (
  user_id      UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  background_notes TEXT NOT NULL DEFAULT '',           -- markdown, user-authored
  stated_goals TEXT NOT NULL DEFAULT '',              -- markdown, user-authored
  preferences  JSONB NOT NULL DEFAULT '{}':::jsonb,
  accessibility JSONB NOT NULL DEFAULT '{}':::jsonb,
```

```

    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TRIGGER trg_user_profiles_updated_at BEFORE UPDATE ON user_profiles
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

preferences JSONB shape (documented for consumers, not enforced by schema):

```

{
  "default_session_minutes": 90,
  "explanation_stance": "standard",
  "voice_enabled": false,
  "tutor_voice_id": null,
  "lecturer_voice_id": null,
  "review_daily_target": 15
}

```

accessibility JSONB shape:

```

{
  "reduce_motion": false,
  "high_contrast": false,
  "font_scale": 1.0,
  "screen_reader_hints_verbose": false,
  "keyboard_shortcuts_enabled": true
}

```

The schema does not validate these blobs. The FastAPI layer holds Pydantic models that do, and only sanitized structures are written back. If a preference key is added, no migration is needed; if a preference key is removed, a data migration cleans up existing rows.

auth_sessions

```

CREATE TABLE auth_sessions (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  TEXT NOT NULL UNIQUE,    -- SHA-256 of the session token
  user_agent  TEXT,
  ip_address  INET,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  expires_at  TIMESTAMPTZ NOT NULL,
  revoked_at  TIMESTAMPTZ
);

CREATE INDEX idx_auth_sessions_user_active
  ON auth_sessions (user_id) WHERE revoked_at IS NULL;
CREATE INDEX idx_auth_sessions_expires
  ON auth_sessions (expires_at) WHERE revoked_at IS NULL;

```

Tokens are hashed on write; the raw token exists only in the cookie sent to the client. A daily job (see §10) purges rows where `expires_at < NOW() - INTERVAL '30 days'`.

6.2. Curriculum: the concept graph

The graph is the pedagogical spine of Studium. It has three primary tables — subjects, concepts, edges — plus a mapping table linking concepts to their source passages and a small metadata table for graph-level annotations.

subjects

```
CREATE TABLE subjects (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),  
  slug        TEXT UNIQUE NOT NULL  
              CHECK (slug ~ '^[a-z0-9][a-z0-9-]{1,63}$'),  
  title       TEXT NOT NULL,  
  short_description TEXT NOT NULL DEFAULT '',  
  long_description TEXT NOT NULL DEFAULT '',      -- markdown, Learner-facing  
  version     INT NOT NULL DEFAULT 1,  
  authored_by UUID REFERENCES users(id) ON DELETE SET NULL,  
  status      artifact_status NOT NULL DEFAULT 'draft',  
  published_at TIMESTAMPTZ,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at  TIMESTAMPTZ  
);  
CREATE TRIGGER trg_subjects_updated_at BEFORE UPDATE ON subjects  
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

version is bumped when the concept graph is substantially reshaped (new nodes added, edges retyped). Point edits to individual concepts do not bump it. Learners are enrolled at a specific graph version through `learner_subjects.subject_version` (§6.5), which is captured at enrollment and used as a drift signal: on session start, if the current `subjects.version` exceeds the learner’s captured version, the frontend offers a migration flow to the new graph.

A candid limitation, and the reason for future work. True graph immutability — a learner pinned to v1 seeing exactly v1’s concepts and edges — is not delivered by this schema. `concepts` and `concept_edges` are not versioned rows; a v1.0 learner still reads today’s concept titles and edges. What is delivered is *drift detection* (`subject_version` mismatch is queryable) and *voluntary migration* (the learner can opt into the new version). This is honest but weaker than a full versioned-graph design. If Studium reaches a scale at which learners are commonly partway through subjects being actively edited (rough trigger: 30+ enrolled learners on a subject undergoing changes), this section is scheduled to be revisited with a proper temporal-table design over `concepts` and `concept_edges`. That is not an MVP concern; it is noted here so no one is surprised when the shortcut surfaces.

concepts

The node table. Each row is one concept in one subject’s graph.

```

CREATE TABLE concepts (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  subject_id  UUID NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
  slug        TEXT NOT NULL
              CHECK (slug ~ '^[a-z0-9][a-z0-9-]{1,80}$'),
  title       TEXT NOT NULL,
  short_description TEXT NOT NULL DEFAULT '',
  long_description TEXT NOT NULL DEFAULT '',      -- markdown
  depth       SMALLINT NOT NULL DEFAULT 1
              CHECK (depth BETWEEN 1 AND 5),
  is_load_bearing BOOLEAN NOT NULL DEFAULT FALSE,
  estimated_minutes SMALLINT NOT NULL DEFAULT 30,
  position     INT NOT NULL DEFAULT 0,           -- sort within subject
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (subject_id, slug)
);

CREATE INDEX idx_concepts_subject_position ON concepts (subject_id,
position);
CREATE INDEX idx_concepts_load_bearing
  ON concepts (subject_id) WHERE is_load_bearing;
CREATE TRIGGER trg_concepts_updated_at BEFORE UPDATE ON concepts
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

depth is the pedagogical difficulty band (1 = introductory, 5 = advanced), used by the Curator to sequence topics and by the Reviewer to weight review cost. is_load_bearing marks the small set of concepts on which most others depend; the frontend's map view weights these visually and the Reviewer prioritizes them. estimated_minutes is the Curator's estimate of how long a first-pass lecture on this concept takes; it feeds session scheduling.

Example rows for the lambda calculus MVP subject:

slug	title	depth	load_bearing	estimated_minutes
syntax	Lambda terms: variables, abstraction, application	1	true	40
alpha-equivalence	Alpha-equivalence and variable capture	2	true	30
beta-reduction	Beta-reduction	2	true	45
church-rosser	The Church-Rosser theorem	4	true	60
church-encoding	Church encodings of data	3	false	50
y-combinator	Fixed-point combinators	4	false	45

concept_edges

Typed directed edges between concepts within a subject.

```

CREATE TYPE concept_edge_kind AS ENUM (
  'prerequisite', -- from is required before to
  'dependency',  -- to uses from
  'generalization', -- to generalizes from
  'application',  -- to is an application of from
  'related'       -- weaker association
);

CREATE TABLE concept_edges (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  subject_id  UUID NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
  from_concept_id UUID NOT NULL REFERENCES concepts(id) ON DELETE CASCADE,
  to_concept_id  UUID NOT NULL REFERENCES concepts(id) ON DELETE CASCADE,
  kind        concept_edge_kind NOT NULL,
  weight       REAL NOT NULL DEFAULT 1.0 CHECK (weight > 0 AND weight <=
1.0),
  note        TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (from_concept_id, to_concept_id, kind),
  CHECK (from_concept_id <> to_concept_id)
);

CREATE INDEX idx_concept_edges_from ON concept_edges (from_concept_id, kind);
CREATE INDEX idx_concept_edges_to ON concept_edges (to_concept_id, kind);
CREATE INDEX idx_concept_edges_subject ON concept_edges (subject_id);

```

Prerequisite edges drive gating: a concept is unlocked when every incoming prerequisite edge's `from_concept_id` has mastery above threshold. Dependency edges drive retrieval and review neighbourhood queries. The distinction matters: a course can dependency-mention a concept from an earlier module without requiring it as a prerequisite.

`weight` is a soft-signal used by the Curator when the graph has multiple viable paths. It defaults to 1.0 (strong) and can be lowered to mark an edge as suggestive rather than required.

Cycle prevention is enforced by application code at graph-authoring time (Alembic data migration validates on subjects publish), not by a database constraint. Postgres cannot cheaply enforce acyclicity on a directed graph without expensive recursive CTEs on every insert.

concept_sources

Links a concept to canonical passages in the corpus that define it, explain it, or provide worked examples. Populated at concept graph authoring time and augmented by the ingestion pipeline.

```

CREATE TYPE concept_source_role AS ENUM (
  'canonical_definition', 'primary_exposition', 'worked_example',
  'exercise', 'historical', 'alternative_stance'
);

```

```

CREATE TABLE concept_sources (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  concept_id  UUID NOT NULL REFERENCES concepts(id) ON DELETE CASCADE,
  source_id   UUID NOT NULL REFERENCES sources(id) ON DELETE CASCADE,
  chunk_ids   UUID[] NOT NULL DEFAULT '{}',
  role        concept_source_role NOT NULL,
  note        TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (concept_id, source_id, role)
);

CREATE INDEX idx_concept_sources_concept ON concept_sources (concept_id,
role);
CREATE INDEX idx_concept_sources_source ON concept_sources (source_id);

```

`chunk_ids` is an array of `source_chunks.id` references (§6.3), not a foreign key column (Postgres arrays can't enforce FK integrity on element level). This is deliberate: the array is a curated pointer set for retrieval to prefer; broken references degrade gracefully rather than blocking a lecture generation. A daily consistency-check job (§10) flags rows with dangling chunk IDs for reviewer attention.

subject_metadata

One-to-one with subjects, carrying computed graph-level statistics maintained by a trigger on concepts and concept_edges.

```

CREATE TABLE subject_metadata (
  subject_id      UUID PRIMARY KEY REFERENCES subjects(id) ON DELETE
CASCADE,
  concept_count   INT NOT NULL DEFAULT 0,
  edge_count      INT NOT NULL DEFAULT 0,
  load_bearing_count INT NOT NULL DEFAULT 0,
  estimated_total_minutes INT NOT NULL DEFAULT 0,
  last_computed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

Maintained by a stored procedure `refresh_subject_metadata(subject_id UUID)` invoked from application code after graph mutations. Not maintained by row-level triggers because MVP-scale bulk imports would fire the trigger thousands of times during ingestion; a single call after the import is cheaper.

6.3. Corpus: source materials

The corpus is the ground truth that agents cite from. Three tables: `sources` (the documents themselves), `source_chunks` (the retrievable units), and their embeddings.

sources

```

CREATE TABLE sources (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  subject_id  UUID NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
  title       TEXT NOT NULL,
  authors     TEXT[] NOT NULL DEFAULT '{}',
  publication_year INT,
  license      license_kind NOT NULL,
  license_notes TEXT,
  uploaded_by UUID REFERENCES users(id) ON DELETE SET NULL,
  storage_path TEXT NOT NULL,           -- fly volume path or S3 key
  content_sha256 CHAR(64) NOT NULL,
  page_count  INT,
  token_count INT,
  ingested_at TIMESTAMPTZ,
  status       artifact_status NOT NULL DEFAULT 'draft',
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at   TIMESTAMPTZ,
  UNIQUE (subject_id, content_sha256)
);

CREATE INDEX idx_sources_subject_status ON sources (subject_id, status)
  WHERE deleted_at IS NULL;
CREATE INDEX idx_sources_uploaded_by ON sources (uploaded_by)
  WHERE uploaded_by IS NOT NULL;
CREATE TRIGGER trg_sources_updated_at BEFORE UPDATE ON sources
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

The (subject_id, content_sha256) unique constraint prevents re-uploading the same textbook twice into the same subject. Cross-subject duplicates are allowed (Pierce's TAPL might reasonably ground both a lambda calculus subject and a type theory subject).

storage_path is opaque to the schema — for MVP it points at a Fly volume mount, later it becomes an R2 object key. The Retrieval spec defines how it is resolved.

license is not a suggestion. A source with license = 'user_uploaded' may only be accessed by learners whose learner_subjects row shows the uploader as themselves (own-uploads only) or by an admin. This is enforced in the query layer, not in Postgres row-level security, because the check depends on session context the database does not hold.

source_chunks

Retrievable units, produced by the ingestion pipeline (Retrieval spec, subsystem 3). This table stores the chunks; embedding computation and search live in that spec.

```

CREATE TABLE source_chunks (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  source_id   UUID NOT NULL REFERENCES sources(id) ON DELETE CASCADE,
  chunk_index INT NOT NULL,

```

```

text          TEXT NOT NULL,
token_count   INT NOT NULL,
page_start    INT,
page_end      INT,
section_path  JSONB NOT NULL DEFAULT '[]'::jsonb, -- e.g. ["Ch 3", "3.2",
"3.2.1"]
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (source_id, chunk_index)
);

CREATE INDEX idx_source_chunks_source ON source_chunks (source_id);

```

section_path is a JSONB array of ordered breadcrumbs. It is stored as JSONB rather than TEXT[] so it can hold hierarchical annotations (chapter number, section title, subsection) without a rigid schema.

source_chunk_embeddings

Separate table, one-to-one with source_chunks, so re-embedding under a new model does not lock the chunk table.

```

CREATE TABLE source_chunk_embeddings (
  chunk_id      UUID PRIMARY KEY REFERENCES source_chunks(id) ON DELETE
CASCADE,
  embedding      VECTOR(1024) NOT NULL,          -- Voyage-3 dimensionality
  model_version  TEXT NOT NULL,                  -- e.g. 'voyage-3.1'
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- HNSW index; parameters tuned in the Retrieval spec.
CREATE INDEX idx_source_chunk_embeddings_hnsw
ON source_chunk_embeddings USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

```

If the embedding model changes, the migration path is: add a new embedding table variant, re-embed in the background, cut over reads once complete, drop the old table. This is documented in the Retrieval spec's model-migration section.

6.4. Content artifacts

Generated content — lecture segments, worked examples, practice problems, rubric prompts, tutorial seed questions. Every artifact is grounded (linked to source chunks), versioned (via superseded_by), and stanced (per §4 of the plan: formal, intuitive, applied, historical).

```

CREATE TYPE artifact_kind AS ENUM (
  'lecture_segment',      -- structured exposition of a concept
  'tutorial_seed',        -- opening question or scenario for tutorial mode
  'worked_example',       -- fully worked demonstration
);

```

```

'practice_problem',      -- problem with hint and solution
'check_question',        -- comprehension check
'model_answer',          -- reference answer for a problem
'rubric_prompt',         -- assessment question shown to learner
'orientation_segment'    -- part of Chapter 0 tour
);

```

```

CREATE TYPE artifact_stance AS ENUM (
  'formal', 'intuitive', 'applied', 'historical', 'default'
);

```

content_artifacts

```

CREATE TABLE content_artifacts (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  concept_id        UUID NOT NULL REFERENCES concepts(id) ON DELETE CASCADE,
  kind              artifact_kind NOT NULL,
  stance            artifact_stance NOT NULL DEFAULT 'default',
  title             TEXT NOT NULL DEFAULT '',
  body              TEXT NOT NULL,           -- markdown
  metadata          JSONB NOT NULL DEFAULT '{}'::jsonb,
  generated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  generated_by      agent_identity NOT NULL,
  model             TEXT,                   -- e.g. 'claude-opus-4-8'
  prompt_hash       CHAR(64),              -- SHA-256 of generation prompt
  cost_usd          NUMERIC(10, 4),
  status            artifact_status NOT NULL DEFAULT 'draft',
  reviewed_by       UUID REFERENCES users(id) ON DELETE SET NULL,
  reviewed_at       TIMESTAMPTZ,
  superseded_by     UUID REFERENCES content_artifacts(id) ON DELETE SET NULL,
  retired_at        TIMESTAMPTZ,
  deleted_at        TIMESTAMPTZ
);

```

```

CREATE INDEX idx_artifacts_concept_kind_active
  ON content_artifacts (concept_id, kind, stance)
  WHERE status = 'active' AND retired_at IS NULL AND deleted_at IS NULL;

```

```

CREATE INDEX idx_artifacts_status
  ON content_artifacts (status)
  WHERE status IN ('draft', 'reviewed') AND deleted_at IS NULL;

```

```

CREATE INDEX idx_artifacts_review_queue
  ON content_artifacts (generated_at)
  WHERE status = 'draft' AND deleted_at IS NULL;

```

-- Foreign-key coverage (§7 rule: every FK indexed)

```

CREATE INDEX idx_artifacts_reviewed_by
  ON content_artifacts (reviewed_by) WHERE reviewed_by IS NOT NULL;

```

```
CREATE INDEX idx_artifacts_superseded_by
ON content_artifacts (superseded_by) WHERE superseded_by IS NOT NULL;
```

The distinction between status and retired_at: status describes lifecycle (draft → reviewed → active → retired), while retired_at is the timestamp at which a previously active artifact was replaced. An artifact can be status = 'retired' with retired_at IS NULL if it was retired without ever going active (rejected in review), or vice versa. The partial index above only serves the common case.

metadata JSONB varies by kind. For practice_problem: {"difficulty": 3, "expected_minutes": 10, "hint_ladder": [...]}. For lecture_segment: {"segment_index": 4, "of": 12, "estimated_minutes": 8}. The Content Ingestion spec (subsystem 5) defines the per-kind schemas; this table does not enforce them.

content_citations

Links from artifacts to the corpus passages that ground them. Every non-trivial factual claim in a generated lecture must be traceable through this table to a source_chunks row.

```
CREATE TABLE content_citations (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  artifact_id UUID NOT NULL REFERENCES content_artifacts(id) ON DELETE
CASCADE,
  source_chunk_id UUID NOT NULL REFERENCES source_chunks(id) ON DELETE
RESTRICT,
  quoted_span  TEXT,                -- the exact substring cited, if
any
  note         TEXT,                -- why this citation supports the
artifact
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (artifact_id, source_chunk_id)
);

CREATE INDEX idx_content_citations_artifact ON content_citations
(artifact_id);
CREATE INDEX idx_content_citations_chunk ON content_citations
(source_chunk_id);
```

ON DELETE RESTRICT on source_chunk_id: if a source chunk would be deleted while artifacts still cite it, the delete is blocked. Handling this is the responsibility of the re-ingestion workflow in the Retrieval spec; the schema refuses to silently orphan citations.

rubric_criteria

The mastery-assessment rubric per concept. Analogous to the existing draft's RubricCriterion model, adapted to the new architecture.

```
CREATE TABLE rubric_criteria (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  concept_id  UUID NOT NULL REFERENCES concepts(id) ON DELETE CASCADE,
  slug        TEXT NOT NULL,
```

```

    prompt          TEXT NOT NULL,          -- the question shown to the
Learner
    key_points       JSONB NOT NULL,         -- array of {point, weight,
source_ref?}
    weight          SMALLINT NOT NULL CHECK (weight IN (1, 2, 3)),
    min_words       SMALLINT NOT NULL DEFAULT 20,
    status          artifact_status NOT NULL DEFAULT 'draft',
    reviewed_by     UUID REFERENCES users(id) ON DELETE SET NULL,
    reviewed_at     TIMESTAMPTZ,
    retired_at      TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (concept_id, slug)
);

CREATE INDEX idx_rubric_criteria_concept_active
  ON rubric_criteria (concept_id)
  WHERE status = 'active' AND retired_at IS NULL;
CREATE INDEX idx_rubric_criteria_reviewed_by
  ON rubric_criteria (reviewed_by) WHERE reviewed_by IS NOT NULL;
CREATE TRIGGER trg_rubric_criteria_updated_at BEFORE UPDATE ON
rubric_criteria
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

key_points shape:

```

[
  {"point": "The application (M N) reduces (M N)[x := N] when M = (λx. M)",
"weight": 3, "source_chunk_id": "..."},
  {"point": "Reduction is undefined when x is bound in M", "weight": 2}
]

```

6.5. Learner state: enrollments and mastery

Two tables carry the per-learner-per-subject state: the enrollment (`learner_subjects`), and the per-concept mastery estimate (`concept_mastery`), backed by an append-only `mastery_events` history that the mastery estimate is derivable from.

`learner_subjects`

```

CREATE TABLE learner_subjects (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  subject_id  UUID NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
  subject_version INT NOT NULL,          -- captured at enrollment
  enrolled_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  current_focus_concept_id UUID REFERENCES concepts(id) ON DELETE SET NULL,
  syllabus_plan JSONB NOT NULL DEFAULT '[]'::jsonb, -- ordered concept
ids

```

```

intake_summary    TEXT NOT NULL DEFAULT '',
preferences       JSONB NOT NULL DEFAULT '{} '::jsonb,
archived_at       TIMESTAMPTZ,
updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (user_id, subject_id)
);

CREATE INDEX idx_learner_subjects_user_active
  ON learner_subjects (user_id) WHERE archived_at IS NULL;
CREATE INDEX idx_learner_subjects_subject
  ON learner_subjects (subject_id);
CREATE INDEX idx_learner_subjects_focus
  ON learner_subjects (current_focus_concept_id)
  WHERE current_focus_concept_id IS NOT NULL;
CREATE TRIGGER trg_learner_subjects_updated_at BEFORE UPDATE ON
learner_subjects
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

subject_version freezes the graph version at enrollment, so a learner three weeks into a subject is not disoriented by a mid-course reshuffling. When the graph is republished, existing enrollments carry a graph_migration_needed flag in preferences and the learner is offered the option to migrate on next session start.

syllabus_plan is the Curator's ordered list of concept IDs — a plan, not a schedule. It is regenerated as mastery evolves; the current plan is stored for auditability and to render the roadmap view.

preferences here overrides user_profiles.preferences at the per-subject level (e.g., a learner might want a slower pace for one subject and a normal one for another).

concept_mastery

The canonical per-concept mastery state. One row per (enrollment, concept) pair. The primary field is p_known, a Bayesian Knowledge Tracing posterior.

```

CREATE TABLE concept_mastery (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  learner_subject_id  UUID NOT NULL REFERENCES learner_subjects(id) ON
DELETE CASCADE,
  concept_id         UUID NOT NULL REFERENCES concepts(id) ON DELETE
CASCADE,
  p_known            REAL NOT NULL DEFAULT 0.1
CHECK (p_known >= 0.0 AND p_known <= 1.0),
  p_known_decayed    REAL NOT NULL DEFAULT 0.1
CHECK (p_known_decayed >= 0.0 AND p_known_decayed <=
1.0),
  evidence_count     INT NOT NULL DEFAULT 0,
  last_evidence_at   TIMESTAMPTZ,
  first_reached_mastery_at TIMESTAMPTZ,
  bkt_params         JSONB NOT NULL DEFAULT '{} '::jsonb,

```

```

    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (learner_subject_id, concept_id)
);

CREATE INDEX idx_concept_mastery_learner
  ON concept_mastery (learner_subject_id, p_known DESC);
CREATE INDEX idx_concept_mastery_stale
  ON concept_mastery (last_evidence_at)
  WHERE last_evidence_at IS NOT NULL;
CREATE TRIGGER trg_concept_mastery_updated_at BEFORE UPDATE ON
concept_mastery
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

p_known is the raw BKT posterior after the most recent evidence update. p_known_decayed is that value decayed by an exponential forgetting curve based on last_evidence_at; it is what the Curator and Reviewer actually read when making scheduling decisions. Storing both means the raw estimate is preserved and the decayed value can be recomputed on read or refreshed by a scheduled job.

Mastery threshold for gating (used by prerequisite checks): p_known_decayed >= 0.85. This value is defined in application code (studium.mastery.MASTERY_THRESHOLD), not hardcoded in the database, so it can be tuned without a migration.

bkt_params holds the per-concept BKT parameters (P(init), P(transit), P(slip), P(guess)):

```
{"p_init": 0.1, "p_transit": 0.15, "p_slip": 0.1, "p_guess": 0.2}
```

Defaults come from concepts.metadata at row creation. Per-learner parameter tuning is a future refinement; the schema supports it by holding parameters on the per-learner row.

mastery_events

Append-only history. Every write to concept_mastery.p_known is preceded by a mastery_events insert, in the same transaction.

```

CREATE TYPE mastery_event_kind AS ENUM (
  'lecture_check_correct', 'lecture_check_incorrect',
  'tutorial_turn_success', 'tutorial_turn_stuck', 'tutorial_turn_recovery',
  'practice_correct', 'practice_incorrect', 'practice_partial',
  'assessment_scored',
  'review_correct', 'review_incorrect',
  'manual_adjustment', 'decay_refresh'
);

CREATE TABLE mastery_events (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  concept_mastery_id  UUID NOT NULL REFERENCES concept_mastery(id) ON
DELETE CASCADE,
  session_id        UUID REFERENCES learning_sessions(id) ON DELETE SET
NULL,
  kind              mastery_event_kind NOT NULL,

```

```

    p_known_before      REAL NOT NULL,
    p_known_after       REAL NOT NULL,
    evidence             JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at           TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_mastery_events_mastery_time
  ON mastery_events (concept_mastery_id, created_at DESC);
CREATE INDEX idx_mastery_events_session
  ON mastery_events (session_id) WHERE session_id IS NOT NULL;

```

evidence shape varies by kind. For practice_incorrect: {"artifact_id": "...", "learner_response": "...", "expected_key_points": [...], "missed_points": [...]}. For assessment_scored: {"attempt_id": "...", "criterion_id": "...", "score": 1}.

The append-only property is enforced by revoking UPDATE and DELETE on this table from the application role in migration 0003_grants.sql. Only the admin role can rewrite history, and only for correcting genuinely bad data (documented in the audit log).

6.6. Learning sessions and turns

A learning_session is one continuous working period. It has a mode (lecture, tutorial, lab, review, office hours, summative assessment), a focus concept, a target duration, and a chronology of turns. Every LLM call inside a session produces a session_turns row.

learning_sessions

```

CREATE TABLE learning_sessions (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
  learner_subject_id  UUID NOT NULL REFERENCES learner_subjects(id) ON
DELETE CASCADE,
  mode              session_mode NOT NULL,
  focus_concept_id   UUID REFERENCES concepts(id) ON DELETE SET NULL,
  target_duration_minutes SMALLINT NOT NULL DEFAULT 90,
  started_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  ended_at           TIMESTAMPTZ,
  end_reason         TEXT
  CHECK (end_reason IN ('completed', 'time_up',
'learner_stop',
                        'idle_timeout', 'error') OR
end_reason IS NULL),
  summary            TEXT,
  key_points         JSONB NOT NULL DEFAULT '[]'::jsonb,
  open_threads       JSONB NOT NULL DEFAULT '[]'::jsonb,
  total_cost_usd     NUMERIC(10, 4) NOT NULL DEFAULT 0
);

```

```

CREATE INDEX idx_learning_sessions_user_time
  ON learning_sessions (user_id, started_at DESC);
CREATE INDEX idx_learning_sessions_active
  ON learning_sessions (user_id) WHERE ended_at IS NULL;
CREATE INDEX idx_learning_sessions_learner_subject
  ON learning_sessions (learner_subject_id, started_at DESC);
CREATE INDEX idx_learning_sessions_focus_concept
  ON learning_sessions (focus_concept_id)
  WHERE focus_concept_id IS NOT NULL;

```

summary, key_points, and open_threads are populated by the Orchestrator at session close (they support the cross-session persistence you specified: a summary plus a brief retrieval check on next session start). Written once, read on every subsequent session start.

total_cost_usd is denormalized from the sum of the session's costs for O(1) read of "how much did this session cost." Maintained by a trigger on agent_traces insert (v1.0 specified session_turns, which has no cost column — a defect corrected here; see §17). The trigger joins through session_turns.session_id to update the correct session's total:

```

CREATE OR REPLACE FUNCTION accumulate_session_cost() RETURNS TRIGGER AS $$
BEGIN
  UPDATE learning_sessions
  SET total_cost_usd = total_cost_usd + NEW.cost_usd
  WHERE id = (SELECT session_id FROM session_turns WHERE id =
NEW.session_turn_id);
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_agent_traces_accumulate_cost
  AFTER INSERT ON agent_traces
  FOR EACH ROW EXECUTE FUNCTION accumulate_session_cost();

```

Non-agent-trace costs (content generation, ingestion, summarisation, grading) are not attributed to a single session and are captured in cost_ledger roll-ups directly rather than through this trigger. See §6.12.

session_turns

The message log. Every LLM interaction, every learner input, every tool invocation.

```

CREATE TABLE session_turns (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  session_id  UUID NOT NULL REFERENCES learning_sessions(id) ON DELETE
CASCADE,
  turn_index  INT NOT NULL,
  actor       agent_identity NOT NULL,
  input       JSONB NOT NULL DEFAULT '{}'::jsonb,
  output      JSONB NOT NULL DEFAULT '{}'::jsonb,
  primitive   TEXT,          -- 'explain_differently', 'prove_it_to_me',

```

```

...
concept_id    UUID REFERENCES concepts(id) ON DELETE SET NULL,
artifact_id   UUID REFERENCES content_artifacts(id) ON DELETE SET NULL,
latency_ms    INT,
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (session_id, turn_index)
);

CREATE INDEX idx_session_turns_session_time
  ON session_turns (session_id, turn_index);
CREATE INDEX idx_session_turns_concept
  ON session_turns (concept_id, created_at DESC)
  WHERE concept_id IS NOT NULL;
CREATE INDEX idx_session_turns_artifact
  ON session_turns (artifact_id)
  WHERE artifact_id IS NOT NULL;
CREATE INDEX idx_session_turns_actor
  ON session_turns (session_id, actor);

```

input and output are opaque to the schema; their shape is defined per-actor in the Agent Runtime spec. For a learner turn: {"kind": "utterance", "text": "..."} or {"kind": "primitive", "primitive": "explain_differently"}. For a tutor turn: {"kind": "reply", "text": "...", "next_intent": "await_response"}.

primitive is denormalized from input when the learner invoked one of the tutorial interaction primitives (§5.4 of the plan). Denormalized because queries like “how often does this learner use ‘I’m lost’” are common enough to warrant an indexable column.

agent_traces

The raw LLM call record. One-to-one with any session_turn whose actor is an agent (not learner).

```

CREATE TABLE agent_traces (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  session_turn_id   UUID NOT NULL UNIQUE REFERENCES session_turns(id) ON
DELETE CASCADE,
  agent             agent_identity NOT NULL,
  model             TEXT NOT NULL,
  prompt_messages   JSONB NOT NULL,           -- full messages array as sent
  system_prompt_hash CHAR(64) NOT NULL,
  completion        TEXT NOT NULL DEFAULT '',
  tools_used        JSONB NOT NULL DEFAULT '[]'::jsonb,
  stop_reason       TEXT,
  tokens_in         INT NOT NULL DEFAULT 0,
  tokens_out        INT NOT NULL DEFAULT 0,
  cache_read_tokens INT NOT NULL DEFAULT 0,
  cache_write_tokens INT NOT NULL DEFAULT 0,
  latency_ms        INT NOT NULL,
  cost_usd          NUMERIC(10, 6) NOT NULL,

```

```

    langfuse_trace_id TEXT,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_agent_traces_created ON agent_traces (created_at DESC);
CREATE INDEX idx_agent_traces_model_created ON agent_traces (model,
created_at DESC);

```

prompt_messages is stored as-sent so a session can be replayed exactly. Storing full prompts has privacy and cost implications: retention is 90 days by default (\$10), longer only for sessions flagged for reviewer attention. A retention job deletes rows older than the window.

system_prompt_hash is the SHA-256 of the system prompt at time of call. Because prompts are byte-stable for a given agent-version, this hash lets us cluster traces by prompt version without storing the full prompt on every row.

6.7. Confusion journal

The journal is the running record of things the learner cannot yet do. Two tables: entries (the standing state of a confusion) and events (its history).

journal_entries

```

CREATE TYPE journal_status AS ENUM ('open', 'partial', 'resolved',
'archived');

CREATE TABLE journal_entries (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
    user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    learner_subject_id UUID NOT NULL REFERENCES learner_subjects(id) ON
DELETE CASCADE,
    concept_id        UUID REFERENCES concepts(id) ON DELETE SET NULL,
    first_session_id  UUID REFERENCES learning_sessions(id) ON DELETE SET
NULL,
    status            journal_status NOT NULL DEFAULT 'open',
    summary           TEXT NOT NULL,                -- Learner- or
Tutor-authored
    hypothesis        TEXT NOT NULL DEFAULT '',      -- Tracker's guess
at the gap
    learner_note      TEXT NOT NULL DEFAULT '',      -- Learner-
authored, editable
    origin            TEXT NOT NULL
CHECK (origin IN ('learner_flagged',
'tracker_inferred',
                    'check_failed', 'assessment_gap')),
    first_seen_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_touched_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    resolved_at       TIMESTAMPTZ,

```

```

        updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );

    CREATE INDEX idx_journal_entries_user_open
        ON journal_entries (user_id, last_touched_at DESC)
        WHERE status IN ('open', 'partial');
    CREATE INDEX idx_journal_entries_learner_subject
        ON journal_entries (learner_subject_id, status, last_touched_at DESC);
    CREATE INDEX idx_journal_entries_concept
        ON journal_entries (concept_id) WHERE concept_id IS NOT NULL;
    CREATE INDEX idx_journal_entries_first_session
        ON journal_entries (first_session_id) WHERE first_session_id IS NOT NULL;
    CREATE TRIGGER trg_journal_entries_updated_at BEFORE UPDATE ON
    journal_entries
        FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

summary is the machine- or learner-authored statement of the confusion ("I can compute alpha-equivalence but I can't tell when a variable capture would happen"). hypothesis is the Confusion-Tracker's inference about the underlying gap ("Learner has not internalized the distinction between free and bound variables when the same identifier appears in both roles"). learner_note is a free-form editable field for the learner's own thoughts.

Separating these three fields matters: the summary is what shows on the journal card; the hypothesis is a Tutor-facing prompt aid that should not be presented to the learner verbatim (it can be wrong, and reading it can be dispiriting); the learner_note is the learner's own space.

journal_events

```

CREATE TYPE journal_event_kind AS ENUM (
    'created', 'revisited', 'partially_addressed', 'resolved',
    'reopened', 'archived', 'hypothesis_updated', 'learner_note_added'
);

CREATE TABLE journal_events (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
    entry_id          UUID NOT NULL REFERENCES journal_entries(id) ON DELETE
    CASCADE,
    session_id        UUID REFERENCES learning_sessions(id) ON DELETE SET NULL,
    kind              journal_event_kind NOT NULL,
    note              TEXT NOT NULL DEFAULT '',
    payload            JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_journal_events_entry_time
    ON journal_events (entry_id, created_at DESC);

```

6.8. Assessment

Assessment attempts are the summative counterpart to formative interaction. The rubric criteria they grade against are defined in `rubric_criteria` (§6.4). Each attempt produces one row per criterion in `assessment_responses`.

`assessment_attempts`

```
CREATE TYPE assessment_mode AS ENUM ('formative', 'summative');
CREATE TYPE assessment_trigger AS ENUM (
    'session_close', 'learner_initiated', 'scheduled', 'unit_gate'
);

CREATE TABLE assessment_attempts (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
    user_id           UUID REFERENCES users(id) ON DELETE SET NULL,
    learner_subject_id UUID REFERENCES learner_subjects(id) ON DELETE SET NULL,
    scope_concept_id  UUID REFERENCES concepts(id) ON DELETE SET NULL,
    mode              assessment_mode NOT NULL DEFAULT 'formative',
    triggered_by       assessment_trigger NOT NULL,
    score              REAL CHECK (score >= 0 AND score <= 1),
    passed            BOOLEAN,
    threshold          REAL NOT NULL DEFAULT 0.75,
    proctored         BOOLEAN NOT NULL DEFAULT FALSE,
    started_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    submitted_at       TIMESTAMPTZ,
    graded_at         TIMESTAMPTZ,
    grading_cost_usd   NUMERIC(10, 4),
    overall_feedback   TEXT,
    session_id        UUID REFERENCES learning_sessions(id) ON DELETE SET NULL
);

CREATE INDEX idx_assessment_attempts_user_time
    ON assessment_attempts (user_id, started_at DESC) WHERE user_id IS NOT NULL;
CREATE INDEX idx_assessment_attempts_scope
    ON assessment_attempts (learner_subject_id, scope_concept_id, submitted_at DESC)
    WHERE learner_subject_id IS NOT NULL;
CREATE INDEX idx_assessment_attempts_concept
    ON assessment_attempts (scope_concept_id) WHERE scope_concept_id IS NOT NULL;
CREATE INDEX idx_assessment_attempts_session
    ON assessment_attempts (session_id) WHERE session_id IS NOT NULL;
```

A note on nullability. `user_id` and `learner_subject_id` were NOT NULL in v1.0. They are SET NULL in v1.1 for one specific reason: the retention policy in §10 declares that `assessment_attempts` metadata is retained across user deletion, and it cannot be retained

if the row is cascade-deleted. Anonymizing the owner columns to NULL preserves the aggregate row while removing PII. See §10 for the corrected right-to-erasure procedure and §17 for the reasoning behind this change (finding A2).

The pass computation, per §3, is `score >= threshold`, evaluated in application code (`studium.assessment.compute_pass`), not by the grading agent. `threshold` defaults to 0.75 to match the existing draft; per-subject overrides live in `subject_metadata` if needed later.

assessment_responses

```
CREATE TABLE assessment_responses (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  attempt_id        UUID NOT NULL REFERENCES assessment_attempts(id) ON
DELETE CASCADE,
  rubric_criterion_id  UUID NOT NULL REFERENCES rubric_criteria(id) ON
DELETE RESTRICT,
  criterion_snapshot  JSONB NOT NULL,    -- the criterion's key_points at
time of grading
  prompt_shown        TEXT NOT NULL,
  learner_response    TEXT NOT NULL,
  grade              SMALLINT CHECK (grade IN (0, 1, 2)),
  feedback            TEXT,
  missing_points      JSONB NOT NULL DEFAULT '[]'::jsonb,
  graded_at           TIMESTAMPTZ,
  UNIQUE (attempt_id, rubric_criterion_id)
);

CREATE INDEX idx_assessment_responses_attempt
ON assessment_responses (attempt_id);
CREATE INDEX idx_assessment_responses_criterion
ON assessment_responses (rubric_criterion_id);
```

`criterion_snapshot` captures the rubric criterion's state at the moment of grading. This is what makes historical assessments meaningful even after the rubric is edited: a graded attempt from three weeks ago still shows the criterion the learner was actually assessed against.

6.9. Review schedule (FSRS)

The Reviewer's schedule of upcoming retrievals. Every concept the learner has any evidence for gets a `review_cards` row; the FSRS algorithm updates the row's stability and difficulty after each review.

review_cards

```
CREATE TABLE review_cards (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  learner_subject_id  UUID NOT NULL REFERENCES learner_subjects(id) ON
DELETE CASCADE,
```

```

    concept_id          UUID NOT NULL REFERENCES concepts(id) ON DELETE
CASCADE,
    stability            REAL NOT NULL DEFAULT 1.0,
    difficulty           REAL NOT NULL DEFAULT 5.0,          -- FSRS scale, 1..10
    retrievability       REAL NOT NULL DEFAULT 1.0,          -- computed at last
review
    due_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_reviewed_at    TIMESTAMPTZ,
    reps                INT NOT NULL DEFAULT 0,
    lapses              INT NOT NULL DEFAULT 0,
    state               TEXT NOT NULL DEFAULT 'new'
CHECK (state IN ('new', 'learning', 'review',
'relearning')),
    suspended          BOOLEAN NOT NULL DEFAULT FALSE,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (learner_subject_id, concept_id)
);

CREATE INDEX idx_review_cards_due
ON review_cards (learner_subject_id, due_at)
WHERE NOT suspended;
CREATE INDEX idx_review_cards_state
ON review_cards (learner_subject_id, state)
WHERE NOT suspended;
CREATE INDEX idx_review_cards_concept
ON review_cards (concept_id);
CREATE TRIGGER trg_review_cards_updated_at BEFORE UPDATE ON review_cards
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

review_events

```

CREATE TABLE review_events (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
    card_id             UUID NOT NULL REFERENCES review_cards(id) ON DELETE
CASCADE,
    session_id         UUID NOT NULL REFERENCES learning_sessions(id) ON DELETE
CASCADE,
    artifact_id        UUID REFERENCES content_artifacts(id) ON DELETE SET NULL,
    rating             SMALLINT NOT NULL CHECK (rating IN (1, 2, 3, 4)), --
again/hard/good/easy
    response_text       TEXT NOT NULL DEFAULT '',
    elapsed_seconds     INT,
    stability_before    REAL NOT NULL,
    stability_after     REAL NOT NULL,
    difficulty_before   REAL NOT NULL,
    difficulty_after    REAL NOT NULL,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```
CREATE INDEX idx_review_events_card_time
ON review_events (card_id, created_at DESC);
```

FSRS state transitions are implemented in `studium.review.fsrs`. The schema does not enforce them; the reviewer service is trusted to compute `stability_after`, `difficulty_after`, and `due_at` correctly, and the event log makes miscalculations recoverable.

6.10. Portfolio

The record of substantive work the learner has produced. Every proof written, program compiled, essay drafted, or derivation completed becomes a `portfolio_items` row. Items are signed (hash-chained) so that “did the learner actually do this work” is verifiable.

portfolio_items

```
CREATE TYPE portfolio_item_kind AS ENUM (
    'proof', 'code', 'prose', 'derivation', 'diagram', 'notebook'
);

CREATE TABLE portfolio_items (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
    user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    learner_subject_id UUID NOT NULL REFERENCES learner_subjects(id) ON
DELETE CASCADE,
    concept_id        UUID REFERENCES concepts(id) ON DELETE SET NULL,
    session_id        UUID REFERENCES learning_sessions(id) ON DELETE SET
NULL,
    kind              portfolio_item_kind NOT NULL,
    title             TEXT NOT NULL,
    body              TEXT NOT NULL,           -- markdown or code
    language          TEXT,                   -- for code
    storage_path       TEXT,                   -- for larger artifacts
(notebook exports)
    content_sha256     CHAR(64) NOT NULL,
    parent_item_id     UUID REFERENCES portfolio_items(id) ON DELETE SET
NULL,
    signature          JSONB NOT NULL,         -- manifest chain
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_portfolio_user_created ON portfolio_items (user_id,
created_at DESC);
CREATE INDEX idx_portfolio_concept ON portfolio_items (concept_id, created_at
DESC)
WHERE concept_id IS NOT NULL;
CREATE INDEX idx_portfolio_session ON portfolio_items (session_id)
WHERE session_id IS NOT NULL;
```

```
CREATE INDEX idx_portfolio_parent ON portfolio_items (parent_item_id)
WHERE parent_item_id IS NOT NULL;
```

signature holds a manifest: previous item's SHA-256, this item's SHA-256, session ID, user ID, timestamp, all signed with a server-held key. This gives a tamper-evident chain per learner-subject. The signing key management belongs to the Infrastructure spec.

parent_item_id supports revision chains: a draft essay revised three times is four rows linked by parent references, so the whole history is legible.

6.11. Cross-session memory and summaries

The persistence layer for what “the app remembers about you” across sessions. Fed by session close and read at session start.

session_summaries

Denormalized read-through cache of session closes. Populated once at session end.

```
CREATE TABLE session_summaries (
  session_id      UUID PRIMARY KEY REFERENCES learning_sessions(id) ON
DELETE CASCADE,
  summary         TEXT NOT NULL,
  key_points      JSONB NOT NULL DEFAULT '[]'::jsonb,
  open_threads    JSONB NOT NULL DEFAULT '[]'::jsonb,
  concepts_touched UUID[] NOT NULL DEFAULT '{}',
  generated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  generated_by    agent_identity NOT NULL DEFAULT 'orchestrator',
  cost_usd        NUMERIC(10, 4) NOT NULL DEFAULT 0,
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TRIGGER trg_session_summaries_updated_at BEFORE UPDATE ON
session_summaries
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

Why a separate table rather than columns on learning_sessions: session summaries can be regenerated (e.g., after a Curator prompt improvement), and their generation is a real LLM call with cost that shouldn't fire on every session read. Existence of a row here means “the session has been summarized.” updated_at was omitted in v1.0 despite the regenerability claim; it is present in v1.1 (finding A3).

retrieval_checks

The brief test at session start that verifies whether the prior session's material has stuck.

```
CREATE TABLE retrieval_checks (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  session_id      UUID NOT NULL REFERENCES learning_sessions(id) ON
DELETE CASCADE,
  based_on_session_id UUID NOT NULL REFERENCES learning_sessions(id) ON
```

```

DELETE CASCADE,
  prompts          JSONB NOT NULL,          -- generated retrieval prompts
  responses         JSONB NOT NULL,          -- learner responses
  scores           JSONB NOT NULL,          -- per-prompt {correct,
partial, incorrect}
  overall_score     REAL NOT NULL CHECK (overall_score >= 0 AND
overall_score <= 1),
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_retrieval_checks_session
  ON retrieval_checks (session_id);
CREATE INDEX idx_retrieval_checks_based_on
  ON retrieval_checks (based_on_session_id);
CREATE TRIGGER trg_retrieval_checks_updated_at BEFORE UPDATE ON
retrieval_checks
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

The overall score is fed into mastery_events as review_correct/review_incorrect events with the relevant concept IDs; the check therefore both diagnoses retention and updates mastery. The result is used by the Curator to decide whether the session can proceed to new material or should first revisit yesterday's.

6.12. Operational: cost, audit

cost_ledger

Daily roll-up of LLM cost per user per model, populated by a daily job. Widened in v1.1 to capture every cost source in the system, not only agent_traces. The v1.0 design summed only agent traces and therefore missed content generation, ingestion, summarisation, and grading — plausibly the dominant cost centres at MVP scale, where content is generated once and read many times. Under-counting the cost is under-counting the risk that costs sink the project, which the plan calls the mode-of-failure most likely to end things in weeks 8–12. Getting the ledger right matters.

Cache-write tokens are split by TTL because pricing differs between 5-minute and 1-hour writes; keeping them in one column would silently double-count or under-count depending on which price the job assumes.

user_id is nullable so retained rows can be anonymized in place during the right-to-erasure procedure without deleting the daily aggregate. The unique constraint uses NULLS NOT DISTINCT (Postgres 15+) so anonymized rows do not collide with each other on the (NULL, day, model) key. Rows with user_id IS NULL represent post-erasure aggregates; the daily job never inserts a NULL owner directly.

```

CREATE TABLE cost_ledger (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),

```

```

user_id          UUID REFERENCES users(id) ON DELETE SET NULL,
day              DATE NOT NULL,
model            TEXT NOT NULL,
tokens_in        BIGINT NOT NULL DEFAULT 0,
tokens_out       BIGINT NOT NULL DEFAULT 0,
cache_read_tokens BIGINT NOT NULL DEFAULT 0,
cache_write_5m_tokens BIGINT NOT NULL DEFAULT 0,
cache_write_1h_tokens BIGINT NOT NULL DEFAULT 0,
cost_agent_usd   NUMERIC(12, 4) NOT NULL DEFAULT 0, -- from
agent_traces
cost_content_usd NUMERIC(12, 4) NOT NULL DEFAULT 0, -- from
content_artifacts
cost_ingestion_usd NUMERIC(12, 4) NOT NULL DEFAULT 0, -- from
ingestion_jobs
cost_summary_usd  NUMERIC(12, 4) NOT NULL DEFAULT 0, -- from
session_summaries
cost_grading_usd  NUMERIC(12, 4) NOT NULL DEFAULT 0, -- from
assessment_attempts
cost_usd          NUMERIC(12, 4) GENERATED ALWAYS AS
                  (cost_agent_usd + cost_content_usd +
cost_ingestion_usd + cost_summary_usd + cost_grading_usd) STORED,
session_count     INT NOT NULL DEFAULT 0,
updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE NULLS NOT DISTINCT (user_id, day, model)
);

CREATE INDEX idx_cost_ledger_user_day
ON cost_ledger (user_id, day DESC) WHERE user_id IS NOT NULL;
CREATE INDEX idx_cost_ledger_day
ON cost_ledger (day DESC);
CREATE TRIGGER trg_cost_ledger_updated_at BEFORE UPDATE ON cost_ledger
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

Attribution of non-agent-trace costs to a user follows source-specific rules:

- **content_artifacts.cost_usd**: attributed to the learner in whose session the generation was triggered. Curator-initiated pre-generation (no requesting learner) is attributed to the system synthetic user.
- **ingestion_jobs.cost_usd**: attributed to the uploading user (sources.uploaded_by), or to system for corpus-import jobs.
- **session_summaries.cost_usd**: attributed to the session's owner (learning_sessions.user_id).
- **assessment_attempts.grading_cost_usd**: attributed to the attempt's owner.

These rules are implemented in the daily roll-up job (studio.jobs.cost_rollup) which reads from the source tables and inserts one row per (user_id, day, model). The five cost columns and the GENERATED ALWAYS total ensure the categories are legible without a separate query.

user_budget_caps

Per-user soft limits. Checked on session start; a session that would push a user over the cap raises a warning to the user and, at higher thresholds, blocks new LLM calls until the next day.

```
CREATE TABLE user_budget_caps (  
  user_id          UUID PRIMARY KEY REFERENCES users(id) ON DELETE  
  CASCADE,  
  daily_soft_usd   NUMERIC(10, 2) NOT NULL DEFAULT 5.00,  
  daily_hard_usd   NUMERIC(10, 2) NOT NULL DEFAULT 8.00,  
  monthly_soft_usd NUMERIC(10, 2) NOT NULL DEFAULT 75.00,  
  monthly_hard_usd NUMERIC(10, 2) NOT NULL DEFAULT 100.00,  
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
CREATE TRIGGER trg_user_budget_caps_updated_at BEFORE UPDATE ON  
user_budget_caps  
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

Defaults match the \$75/learner-month soft ceiling with 33% headroom before hard-blocking. Overrides per user (yours, as the reviewer, might reasonably be higher) via UPDATE.

audit_log

Administrative and privileged actions, for accountability. Not a general-purpose event log; only actions that mutate other users' data, retire content, adjust rubric criteria, or override gating.

```
CREATE TABLE audit_log (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),  
  actor_user_id UUID REFERENCES users(id) ON DELETE SET NULL,  
  action      TEXT NOT NULL,  
  target_type TEXT NOT NULL,  
  target_id   UUID,  
  before      JSONB,  
  after       JSONB,  
  reason      TEXT NOT NULL DEFAULT '',  
  ip_address  INET,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_audit_log_actor_time  
  ON audit_log (actor_user_id, created_at DESC);  
CREATE INDEX idx_audit_log_target  
  ON audit_log (target_type, target_id, created_at DESC);
```

Append-only. UPDATE and DELETE revoked from the application role.

6.13. Content ingestion and review

ingestion_jobs

The pipeline tracker for a source being processed into chunks, embeddings, and initial concept_sources mappings. Populated when a source is uploaded; drained by the ingestion worker.

```
CREATE TYPE ingestion_job_kind AS ENUM (
  'extract_text', 'chunk', 'embed', 'suggest_concept_mapping'
);
CREATE TYPE ingestion_job_status AS ENUM (
  'pending', 'running', 'done', 'failed', 'cancelled'
);

CREATE TABLE ingestion_jobs (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  source_id   UUID NOT NULL REFERENCES sources(id) ON DELETE CASCADE,
  kind        ingestion_job_kind NOT NULL,
  status      ingestion_job_status NOT NULL DEFAULT 'pending',
  attempt     SMALLINT NOT NULL DEFAULT 0,
  started_at  TIMESTAMPTZ,
  finished_at TIMESTAMPTZ,
  error       TEXT,
  cost_usd    NUMERIC(10, 4) NOT NULL DEFAULT 0,
  payload     JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_ingestion_jobs_pending
  ON ingestion_jobs (created_at) WHERE status = 'pending';
CREATE INDEX idx_ingestion_jobs_source
  ON ingestion_jobs (source_id, kind, status);
CREATE TRIGGER trg_ingestion_jobs_updated_at BEFORE UPDATE ON ingestion_jobs
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

content_review_queue

Items flagged for human review — you, in MVP. Populated by the Confusion-Tracker when a generated lecture seems to have caused unusual confusion, by the Evaluator when a grading looks suspicious, and by learners via a “report this” affordance.

```
CREATE TYPE review_flag_source AS ENUM (
  'system_confidence', 'learner_report', 'tracker_pattern',
  'evaluator_disagreement', 'random_sample'
);
CREATE TYPE review_status AS ENUM (
  'pending', 'in_review', 'resolved', 'dismissed'
);
```

```

CREATE TABLE content_review_queue (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  artifact_id       UUID REFERENCES content_artifacts(id) ON DELETE CASCADE,
  session_turn_id   UUID REFERENCES session_turns(id) ON DELETE CASCADE,
  source            review_flag_source NOT NULL,
  reason            TEXT NOT NULL,
  severity          SMALLINT NOT NULL DEFAULT 2 CHECK (severity BETWEEN 1 AND
3),
  status            review_status NOT NULL DEFAULT 'pending',
  assigned_to       UUID REFERENCES users(id) ON DELETE SET NULL,
  resolution_note   TEXT,
  resolved_at       TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CHECK (artifact_id IS NOT NULL OR session_turn_id IS NOT NULL)
);

CREATE INDEX idx_review_queue_pending
  ON content_review_queue (severity DESC, created_at)
  WHERE status = 'pending';
CREATE INDEX idx_review_queue_assigned
  ON content_review_queue (assigned_to, status)
  WHERE assigned_to IS NOT NULL;
CREATE TRIGGER trg_review_queue_updated_at BEFORE UPDATE ON
content_review_queue
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

The check constraint requires that at least one target reference is set — a review item that points at neither an artifact nor a session turn has no subject.

7. Indexing strategy

Every index in the schema falls into one of four categories: primary key (automatic), foreign key support (added explicitly), query-shape support (justified below), or partial index (justified by the “active only” pattern).

Foreign key indexes are added on every FK column, without exception. Postgres does not automatically index the referencing side of a foreign key, and unindexed FKs cause expensive sequential scans on delete cascades. A partial index (e.g., `WHERE reviewed_by IS NOT NULL`) is acceptable and is what most nullable FKs use here — it satisfies the same access patterns at lower storage cost. A composite index whose leading column is the FK also counts. The v1.0 spec claimed this rule but violated it on twenty columns; v1.1 fixes those omissions (finding A5) and the §14 CI check verifies the rule against `pg_catalog` for every future migration.

Query-shape indexes are justified by specific access patterns:

Index	Query it supports
-------	-------------------

Index	Query it supports
<code>idx_concepts_subject_position</code>	Curator loading a subject's concept list in order
<code>idx_concept_edges_from</code>	Prerequisite unlock check: "what concepts require this one?"
<code>idx_artifacts_concept_kind_active</code>	Lecturer fetching current lecture segment for concept X, formal stance
<code>idx_concept_mastery_learner</code>	Frontend rendering the mastery-sorted concept list
<code>idx_review_cards_due</code>	Reviewer picking today's due cards for user
<code>idx_journal_entries_user_open</code>	Session-start view of the learner's open confusions
<code>idx_session_turns_concept</code>	Historical context: "what has this learner said about concept X?"
<code>idx_cost_ledger_user_day</code>	Budget check on session start
<code>idx_review_queue_pending</code>	Reviewer's queue view, ordered by severity then age

Partial indexes appear wherever a column has a strongly bimodal distribution (mostly one value) and queries only ever look at the minority side. `deleted_at IS NULL, status = 'active', suspended = FALSE`. These are cheaper to maintain and faster to scan than full indexes.

HNSW index on embeddings is tuned for the retrieval workload: `m = 16` and `ef_construction = 64` are pgvector defaults, appropriate for corpus sizes up to a few million chunks. The Retrieval spec revisits these as the corpus grows.

Indexes not added (deliberately):

- No index on `session_turns.output` JSONB. Full-text search over turn output is not a common query; it would be expensive to maintain and the traces table is time-bounded anyway.
- No index on `agent_traces.prompt_messages`. Same reasoning.
- No index on `learner_note` in `journal_entries`. Learners searching their own notes is a v1.1 feature; when it ships, add a `pg_trgm` GIN index at that time.

8. Query patterns

The most frequent access paths, with SQL. These serve as informal correctness tests for the schema: if a common access path is awkward, the schema is wrong.

Session start: assemble context. Called every time a learner opens the app.

```
-- 1. Learner + active enrollments
SELECT u.*, up.preferences, up.accessibility
```

```

FROM users u
LEFT JOIN user_profiles up ON up.user_id = u.id
WHERE u.id = :user_id AND u.deleted_at IS NULL;

-- 2. Active enrollments with subject metadata
SELECT ls.*, s.title, s.slug, sm.concept_count, sm.load_bearing_count
FROM learner_subjects ls
JOIN subjects s ON s.id = ls.subject_id
LEFT JOIN subject_metadata sm ON sm.subject_id = s.id
WHERE ls.user_id = :user_id AND ls.archived_at IS NULL;

-- 3. Open journal entries for the currently selected enrollment
SELECT je.*
FROM journal_entries je
WHERE je.learner_subject_id = :learner_subject_id
      AND je.status IN ('open', 'partial')
ORDER BY je.last_touched_at DESC
LIMIT 20;

-- 4. Due review cards
SELECT rc.*, c.title, c.slug
FROM review_cards rc
JOIN concepts c ON c.id = rc.concept_id
WHERE rc.learner_subject_id = :learner_subject_id
      AND rc.due_at <= NOW()
      AND NOT rc.suspended
ORDER BY rc.due_at
LIMIT 10;

-- 5. Prior session summary (for retrieval check generation)
SELECT ss.*, session.mode, session.focus_concept_id
FROM session_summaries ss
JOIN learning_sessions session ON session.id = ss.session_id
WHERE session.learner_subject_id = :learner_subject_id
      AND session.ended_at IS NOT NULL
ORDER BY session.ended_at DESC
LIMIT 1;

```

These five queries together run in single-digit milliseconds on a warm cache at MVP scale. Under contention they can be issued in parallel from the FastAPI handler.

Prerequisite unlock check. Called when the Curator considers whether a concept can be attempted.

```

WITH prereqs AS (
  SELECT ce.from_concept_id AS required
  FROM concept_edges ce
  WHERE ce.to_concept_id = :concept_id
        AND ce.kind = 'prerequisite'
)

```

```

SELECT
  COUNT(*) AS required_count,
  COUNT(*) FILTER (WHERE cm.p_known_decayed >= 0.85) AS satisfied_count
FROM prereqs
LEFT JOIN concept_mastery cm
  ON cm.concept_id = prereqs.required
  AND cm.learner_subject_id = :learner_subject_id;

```

A concept is unlocked when `required_count = satisfied_count`. The 0.85 threshold is application-defined; if it changes, this query is regenerated by the ORM layer.

Mastery update after evidence. Called after every graded interaction. Runs in a single transaction.

```

BEGIN;

INSERT INTO mastery_events (
  concept_mastery_id, session_id, kind,
  p_known_before, p_known_after, evidence
) VALUES (:cm_id, :session_id, :kind, :before, :after, :evidence);

UPDATE concept_mastery
SET p_known = :after,
    p_known_decayed = :after, -- recomputed at read time based on age
    evidence_count = evidence_count + 1,
    last_evidence_at = NOW(),
    first_reached_mastery_at = COALESCE(
      first_reached_mastery_at,
      CASE WHEN :after >= 0.85 THEN NOW() ELSE NULL END
    )
WHERE id = :cm_id;

COMMIT;

```

`p_known_decayed` is set equal to `p_known` on write. A separate daily job recomputes decay for all rows; individual reads that need a fresher value can call `mastery.decayed(row)` in application code.

Lecture segment retrieval. Called when the Lecturer needs the current segment for a concept.

```

SELECT ca.*, cc.source_chunk_id, sc.text AS source_text,
       s.title AS source_title, s.authors
FROM content_artifacts ca
LEFT JOIN content_citations cc ON cc.artifact_id = ca.id
LEFT JOIN source_chunks sc ON sc.id = cc.source_chunk_id
LEFT JOIN sources s ON s.id = sc.source_id
WHERE ca.concept_id = :concept_id
  AND ca.kind = 'lecture_segment'
  AND ca.stance = :stance

```

```

AND ca.status = 'active'
AND ca.retired_at IS NULL
AND ca.deleted_at IS NULL
ORDER BY (ca.metadata->>'segment_index')::INT;

```

The Retrieval spec extends this with vector-similarity queries against `source_chunk_embeddings` for on-demand grounding beyond the pre-mapped `concept_sources`.

Cost budget check. Called before starting any billable operation. Reads the widened cost ledger (see §6.12) so the check reflects total spend, not only agent traces.

```

SELECT
  COALESCE(SUM(cost_usd) FILTER (WHERE day = CURRENT_DATE), 0) AS today,
  COALESCE(SUM(cost_usd) FILTER (WHERE day >= date_trunc('month',
CURRENT_DATE)), 0) AS this_month,
  ubc.daily_hard_usd, ubc.monthly_hard_usd
FROM cost_ledger cl
RIGHT JOIN user_budget_caps ubc ON ubc.user_id = :user_id
WHERE cl.user_id = :user_id OR cl.user_id IS NULL
GROUP BY ubc.daily_hard_usd, ubc.monthly_hard_usd;

```

For active-day cost tracking that hasn't rolled up yet, application code additionally sums the day's live cost across all five source tables (`agent_traces`, `content_artifacts` created today, `ingestion_jobs` finished today, `session_summaries` generated today, `assessment_attempts`.grading_cost_usd graded today) for the current user. This is cheap because the day's rows are always in the recent, in-memory index. A helper `studium.cost.today_spent(user_id)` centralizes this logic so no caller assembles it by hand.

Journal entry creation from the Confusion-Tracker.

```

INSERT INTO journal_entries (
  user_id, learner_subject_id, concept_id, first_session_id,
  summary, hypothesis, origin
) VALUES (
  :user_id, :learner_subject_id, :concept_id, :session_id,
  :summary, :hypothesis, 'tracker_inferred'
)
RETURNING id;

INSERT INTO journal_events (entry_id, session_id, kind, note)
VALUES (:entry_id, :session_id, 'created', :note);

```

9. Concurrency and consistency

Studium is not high-concurrency. Three MVP users, tens of concurrent sessions at classroom scale, low hundreds at university scale. The consistency model can be simple.

Default isolation. `READ COMMITTED`. Sufficient for essentially every query in this document.

Mastery updates use REPEATABLE READ. The read-modify-write cycle on `concept_mastery` (read current `p_known`, compute update, write back) must not lose updates. If two agent turns concurrently produce evidence for the same concept, both increments must land. The FastAPI transaction wrapper for `mastery.apply_evidence()` opens a `REPEATABLE READ` transaction and retries on serialization failure with exponential backoff (max 3 attempts). This is preferable to pessimistic row-locking because contention is rare and retries are cheap.

Session turn ordering. `session_turns.turn_index` is unique per session. The FastAPI orchestrator assigns turn indices from an in-memory per-session counter; on process crash the counter reinitializes from `MAX(turn_index) + 1`. This is safe because a session is served by exactly one process at a time (session affinity in the load balancer).

Ingestion job claim. Workers claim pending jobs with `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1`, standard pattern for job queues on Postgres. `SKIP LOCKED` requires Postgres 9.5+, which is well below our floor of 16.

Optimistic writes for content authoring. When you edit a concept or artifact, the client submits the row's `updated_at` as an if-match condition. If a background regeneration has modified the row in the meantime, the write is rejected with a conflict response and the client re-fetches. Prevents overwriting each other's edits.

Transaction boundaries. Every FastAPI request handler opens exactly one transaction. Long-running operations (LLM calls) commit any state-mutating writes before the call and start a new transaction after. A dropped connection during an LLM call therefore does not lose already-written state. Cost writes (`agent_traces`) happen after the call completes; if that write fails, a warning is logged and the call's cost is reconciled by a daily job that compares the trace count to invoice line items from Anthropic.

10. Data lifecycle and retention

Every table has a defined retention policy. A weekly job (`studium.jobs.retention`) applies them.

Table	Retention	Deletion policy
<code>users</code>	Indefinite	Soft delete on account close; hard delete after 30 days if no dispute
<code>auth_sessions</code>	30 days after <code>expires_at</code>	Hard delete
<code>learning_sessions</code>	2 years	Hard delete, cascades to turns
<code>session_turns</code>	2 years (cascade)	Cascade from session
<code>agent_traces</code>	90 days	Hard delete; retained longer only if linked to a review queue item

Table	Retention	Deletion policy
mastery_events	5 years	Hard delete (mastery estimate persists in concept_mastery)
journal_events	Indefinite	Kept for full journal history
journal_entries	Indefinite while any status	Deleted on learner request
review_events	2 years	Hard delete
assessment_attempts	Indefinite	Kept as portfolio evidence
assessment_responses	Indefinite	Kept as portfolio evidence
portfolio_items	Indefinite	Kept until learner deletion
cost_ledger	3 years	Hard delete
audit_log	7 years	Hard delete (regulatory margin)
ingestion_jobs	90 days after completion	Hard delete
content_review_queue	1 year after resolution	Hard delete
sources uploaded content	Retained while any citation exists	Then hard delete

Right to erasure (PIPEDA). The v1.0 procedure was cascade-unsafe: it deleted learner_subjects in an early step, which cascades to assessment_attempts and other rows that a later step said to retain. The result would have been that of the four things v1.0 said to keep, only the audit log survived. This is finding A2.

The v1.1 procedure inverts the order — anonymize first, then delete — and depends on the schema change (§6.8, §6.12) that made assessment_attempts.user_id, assessment_attempts.learner_subject_id, and cost_ledger.user_id nullable with ON DELETE SET NULL. Retained rows survive the eventual cascade because their owner columns already point at NULL by the time the delete happens.

When a user requests account deletion:

1. **Mark closed.** Set users.deleted_at = NOW(). Insert an audit_log row recording the erasure request.
2. **Anonymize retained aggregates**, in the same transaction:
 - UPDATE assessment_attempts SET user_id = NULL, learner_subject_id = NULL WHERE user_id = :uid
 - UPDATE cost_ledger SET user_id = NULL WHERE user_id = :uid
 - UPDATE audit_log SET actor_user_id = NULL WHERE actor_user_id = :uid (except the erasure-request row itself, which is retained with actor for accountability, then anonymized after the retention window)
 - UPDATE assessment_responses SET learner_response = '[redacted]' WHERE attempt_id IN (SELECT id FROM assessment_attempts WHERE

`user_id IS NULL AND submitted_at > NOW() - INTERVAL '1 hour')` — a narrow window is used so responses that had a real owner at the time of this transaction are the only ones redacted.

3. **Purge learner-owned data.** Delete `learner_subjects` WHERE `user_id = :uid`, which cascades to `concept_mastery`, `learning_sessions` (which cascades further to `session_turns`, `agent_traces`, `session_summaries`, `retrieval_checks`), `journal_entries` (cascading to `journal_events`), `review_cards` (cascading to `review_events`), `portfolio_items`. Also delete `user_profiles` WHERE `user_id = :uid` and `auth_sessions` WHERE `user_id = :uid`.
4. **Grace window.** After 30 days without dispute (checked by a scheduled job), hard-delete the users row itself. The 30-day window handles accidental deletion and account recovery. The remaining anonymized rows in `assessment_attempts`, `cost_ledger`, `audit_log` persist beyond this — they carry no PII and are used only for system-level metrics.

Two subtleties worth calling out:

Why not use a “tombstone user” instead of nullable columns? A shared sentinel user (one row that every anonymized reference points at) is marginally cleaner in the schema but creates a hotspot for aggregation queries and makes the anonymized state harder to identify. Nullable columns with a partial index (`WHERE user_id IS NOT NULL`) achieve the same anonymization with better query characteristics.

Order matters within step 2. The `assessment_responses` redaction depends on `assessment_attempts.user_id` still being the deleted user, so it must run before the anonymize update on `assessment_attempts`. In practice, either sequence the redaction first or use a subquery keyed off a captured user ID rather than the “just-nulled” state — the implementation in `studium.privacy.erase_user` uses the former.

Backup posture. Fly.io managed Postgres provides continuous WAL archival and point-in-time recovery within a 7-day window. This is sufficient for MVP. When institutional users come in, add nightly encrypted backups to a second region (spec’d in Infrastructure).

Uploaded sources. Deleted from storage when the last sources row referencing them is deleted. A weekly reconciliation job compares `sources.storage_path` values against the storage bucket and reports orphaned objects.

11. Privacy, security, and access control

Access control model. Three roles: learner, reviewer, admin. Enforcement lives in the FastAPI middleware, not in database row-level security. Reasons: RLS in Postgres requires setting session variables at connection time, which is awkward under PgBouncer transaction pooling; and the check logic is cheap to run in application code where the current user is naturally in scope.

Access rules.

- A learner can read and write only their own users, user_profiles, learner_subjects, concept_mastery, journal_entries, learning_sessions, session_turns, assessment_attempts, assessment_responses, review_cards, portfolio_items, retrieval_checks rows.
- A learner can read published subjects, concepts, concept_edges, active content_artifacts for subjects they are enrolled in, rubric_criteria (but not key_points), sources metadata (title, authors), and source_chunks in service of citations shown to them.
- A learner cannot read agent_traces (even their own — the raw prompts are internal).
- A learner cannot read rubric_criteria.key_points at any time. This mirrors the existing draft's principle that rubric key points never leave the server.
- A reviewer can read all of the above across users, plus content_review_queue, all content_artifacts regardless of status, all rubric_criteria including key_points, and agent_traces for reviewer-flagged sessions.
- An admin can additionally mutate any row and read audit_log.

Data classification.

- **Public:** subject, concept, edge, published artifact metadata.
- **Learner-private:** everything in a learner's own row set as listed above.
- **System-internal:** agent_traces, rubric_criteria.key_points, content_artifacts.status = 'draft' bodies, ingestion payloads.
- **Audit-only:** audit_log.

Encryption at rest. Fly.io's managed Postgres encrypts storage volumes by default. sources.storage_path payloads on the Fly volume are similarly encrypted at the volume level. Additional per-column encryption is not needed at MVP; if institutional deployment brings requirements around it, revisit.

Encryption in transit. All Postgres connections require TLS. Enforced by the Fly.io Postgres configuration and asserted by application startup checks.

Secret handling. No secrets stored in the database. Anthropic API key, session signing key, portfolio signing key all live in Fly.io secrets, accessed via environment variables.

Password hashing. Argon2id, parameters set to at least OWASP 2025 recommendations. Password reset tokens are single-use, expire in 1 hour, hashed on storage.

Prompt injection defense at the data boundary. Any user-uploaded text (source uploads, learner notes, journal entries) that is included in an LLM prompt is wrapped in a demarcation block (<user_content>...</user_content>) with an explicit system-prompt instruction that content inside is not to be executed as instructions. This is agent-spec territory but noted here because the data layer is what stores that content and the boundary is enforced at every read into an agent context.

12. Migration from the existing draft

The draft persists in JSON files under `coursepack/`, `progress/`, and `per-unit pack` files. A one-time migration script (`scripts/migrate_from_draft.py`) transforms them into the new schema. The migration is not required for shipping — MVP can be built greenfield — but running it lets you carry your lambda calculus draft-in-progress forward as the seed subject.

Mapping.

Draft artifact	New schema target
<code>Course.name</code> , <code>Course.title</code>	subjects row
<code>Module</code>	Not preserved as a first-class entity; modules become a grouping of concepts via a nullable <code>concepts.module_slug</code> column added in migration <code>0002_module_slug.sql</code>
<code>Unit</code>	concepts row (each unit becomes a concept node)
<code>UnitPack.overview</code>	<code>concepts.short_description</code>
<code>UnitPack.study_guide.summary</code> , <code>.why_it_matters</code>	<code>concepts.long_description</code> (assembled)
<code>UnitPack.learning_objectives</code>	<code>content_artifacts</code> rows, <code>kind = lecture_segment</code> , one per segment
<code>UnitPack.key_definitions</code>	<code>content_artifacts</code> , <code>kind = worked_example</code> with <code>metadata.kind = 'definition'</code>
<code>UnitPack.segments[]</code> .content	<code>content_artifacts</code> , <code>kind = lecture_segment</code> , <code>metadata.segment_index</code> set
<code>UnitPack.segments[]</code> .practice	<code>content_artifacts</code> , <code>kind = practice_problem</code> + separate <code>model_answer</code> artifact
<code>RubricCriterion</code>	<code>rubric_criteria</code> row
<code>Progress.attempts</code>	<code>assessment_attempts</code> + <code>assessment_responses</code> rows
<code>progress[unit]</code> .per_criterion	derived into <code>mastery_events</code> (<code>kind = assessment_scored</code>)
Passed status	derived (not stored); recomputed from <code>assessment_attempts.passed</code>

Sources. Draft PDFs under `material/` are copied into the Fly volume, hashed, and inserted as sources rows with `license = 'public_domain'` where the source is genuinely public-domain (Church papers), and `license = 'permission_granted'` for anything else with a note in `license_notes` describing the basis. You review the migration output before publishing.

What's not migrated. No historical `agent_traces` (draft has none), no `journal_entries` (concept didn't exist), no `review_cards` (FSRS wasn't in use). These populate as the new system runs.

Order of operations. Migration runs against a fresh database, populates the schema, then Alembic runs its migrations on top. This is unusual but correct here because we're seeding rather than migrating between versions.

13. Migration tooling and workflow

Alembic setup. Under `backend/migrations/`. Autogenerate is used cautiously — every generated migration is reviewed by hand before commit because autogenerate misses check constraints, comment changes, and enum evolution. Naming convention: `NNNN_verb_object.py`, e.g., `0007_add_review_cards_suspended.py`.

Migration types and rules.

- **Additive migrations** (new tables, new nullable columns, new indexes) run against the live database with no application downtime.
- **Non-additive migrations** (drop column, drop table, change type, rename) require a two-step deploy: (1) ship code that reads/writes both old and new shape, (2) ship the migration, (3) ship code that reads/writes only the new shape. No exceptions at institutional scale; at MVP scale with three users, taking a maintenance window is acceptable and simpler.
- **Enum evolution.** Adding a value: `ALTER TYPE ... ADD VALUE 'new_value', safe`. Removing or renaming: requires the two-step dance because `ALTER TYPE ... RENAME VALUE` still requires no running query to reference the old value. Prefer adding new values and deprecating old ones over renames.
- **Data migrations** are separate from schema migrations. A schema change and a backfill of the new column live in different Alembic revisions, so the backfill can be rerun on failure without re-applying the schema change.

CI checks.

1. Every migration has a corresponding downgrade. `test_migrations_have_downgrade` asserts this.
2. Every migration runs cleanly against a fresh database. `test_full_migration_sequence` builds a Postgres from empty to head.
3. Every migration is reversible. `test_migration_round_trip` runs `upgrade head`, `downgrade -1`, `upgrade head` and asserts no diff in `pg_dump --schema-only` output.
4. No migration is committed without a corresponding entry in `CHANGELOG.md` describing the semantic change.

Local development. `make db-reset` drops and recreates the dev database, then runs migrations plus a seed script that populates a demo user, the lambda calculus subject, and 10 sample concepts.

14. Testing strategy

The test suite is tiered so that most correctness checks run without a live database. The v1.0 spec assumed schema tests would read `information_schema`, which needs a running server; v1.1 reframes this because the dev's build correctly identified that the same truths can be asserted one step earlier — against SQLAlchemy metadata and Alembic migration source — with the practical consequence that most tests run in under a second in CI. The offline tier is what caught the twenty unindexed foreign keys in v1.0 and it is what will catch the twenty-first.

Tier 1 — Offline (no database required). In `backend/tests/schema/` and `backend/tests/unit/`:

- Foreign key coverage: every FK column has a leading or partial index. Verified against SQLAlchemy metadata.
- Every mutable table has an `updated_at` trigger declared in a migration file. Verified by AST inspection of migration sources.
- Every soft-delete table has a partial index excluding deleted rows.
- Enum values are stable — a snapshot test over the 18 enum types fails on unreviewed changes.
- Every migration has a real downgrade (not a stub) and a `CHANGELOG.md` entry. AST-verified.
- Deterministic rules: BKT update math, forgetting-curve decay, `compute_pass` aggregation, FSRs rating transitions, access-projection column stripping. Standard unit tests.
- Static verification: SQL DDL compiles against the real postgresql dialect; ruff clean; every module imports.

Tier 2 — Online (requires Postgres 16 + pgvector). In `backend/tests/integrity/` and `backend/tests/queries/`:

- Cascade behavior: deleting a `concept_mastery` row cascades to `mastery_events`; `content_citations` blocks `source_chunks` deletion; `assessment_responses` uniqueness holds on (`attempt_id`, `rubric_criterion_id`).
- The `uuid_generate_v7()` fallback produces valid 32-char UUIDs that round-trip through the `::uuid` cast.
- Cycle detection at subject-publish time catches $A \rightarrow B \rightarrow A$ prerequisite graphs.
- The right-to-erasure procedure leaves retained rows intact and cascades everything else, tested against a seeded user with data in every table.
- Query-shape tests: the §8 access paths use the expected indexes (`EXPLAIN ANALYZE` output snapshotted; plan shape asserted, wall-clock loosely bounded).
- Migration round-trip: `upgrade head, downgrade -1, upgrade head` produces no diff in `pg_dump --schema-only`.
- HNSW index creation against pgvector.

Fixture data. `backend/tests/fixtures/lambda.py` builds a minimal lambda calculus subject with 6 concepts, 8 edges, 1 source, 20 chunks. Reused by all downstream tests.

CI wiring. Tier 1 gates every push. Tier 2 runs on the main branch and on pull requests that touch schema, migrations, or the `studium.privacy` and `studium.cost` modules. The Docker Compose file at `backend/docker-compose.test.yml` provisions a pinned `postgres:16` image with `pgvector` preinstalled.

15. Deployment considerations

Sizing at MVP scale (3 users).

- Fly.io managed Postgres: 1 CPU, 1GB RAM, 10GB disk. This is well over-provisioned but the price difference below this tier is negligible.
- Backup retention: 7 days (Fly.io default).
- Regions: primary in `yyz` (Toronto), no replica.
- PgBouncer: not needed at 3 users; direct connections from the FastAPI process are fine. Add PgBouncer when concurrent connections exceed 20.

Sizing at classroom scale (30 users).

- 2 CPU, 4GB RAM, 25GB disk.
- Add PgBouncer.
- Nightly logical backup to R2 in addition to Fly.io's WAL archival.

Sizing at university scale (500 users).

- 4 CPU, 16GB RAM, 100GB disk.
- Read replica in Toronto region for reporting queries.
- HNSW index parameters revisited; corpus embeddings likely need $m = 32$.
- Consider partitioning `agent_traces` and `session_turns` by month.

None of the sizing decisions require schema changes. The schema is scale-agnostic within the range Studium plausibly needs.

16. Open questions and forward references

Items this spec deliberately punts to later specs. Recorded here so nothing is lost.

- **Agent Runtime spec (subsystem 2).** The `agent_traces.system_prompt_hash` values need a registry that maps `hash` $\rightarrow$ `prompt version` $\rightarrow$ `agent version`. That registry lives in the agent runtime code, not the data layer, but the join is bidirectional.
- **Retrieval spec (subsystem 3).** Chunking strategy, embedding model versioning, hybrid search ranking, and reranker configuration all touch this schema. Expected

additions: a `source_chunks.rerank_metadata` JSONB column, possibly a separate `chunk_relations` table for parent-child chunk hierarchies. Both are additive.

- **Content Ingestion spec (subsystem 5).** The per-kind `content_artifacts.metadata` shapes are defined there. This spec commits only that the column exists and is JSONB.
- **Evaluation spec (subsystem 6).** A `golden_datasets` table for regression test fixtures may be needed, populated from human-reviewed session traces. Anticipated but not spec'd.
- **Infrastructure spec (subsystem 7).** Signing key management for portfolio manifests. Backup rotation. Point-in-time recovery test cadence.
- **BKT parameter defaults.** This spec allows per-concept BKT parameters but does not specify defaults or a tuning procedure. That lives in the Agent Runtime spec's mastery-model section.
- **FSRS parameter defaults.** Same story. The default FSRS parameter set is defensible for a general-purpose deployment; per-learner tuning is a v1.1 feature.
- **Concept graph authoring UI.** You will build the lambda calculus graph directly against the schema (a seed script). The authoring UI is a separate concern, likely in the Content Ingestion spec.

17. Version history

v1.1 — 14 August 2026. Folds in the corrections identified by the subsystem-1 build. Every change below traces to a finding in the build review (`spec/01-data-layer-review.md`) or to a decision recorded in the change memo that accompanies this version.

Blocking corrections (defects that would have prevented the schema from being created).

Finding	Change
A1	Rewrote <code>uuid_generate_v7()</code> fallback to produce exactly 32 hex chars (v1.0 produced 33). Removed the incorrect attribution to <code>uuid-oss</code> and pointed the preferred path at <code>pg_uuidv7</code> . §4, §6.0.
A2	Rewrote the right-to-erasure procedure to be cascade-safe. Made <code>assessment_attempts.user_id</code> , <code>assessment_attempts.learner_subject_id</code> , and <code>cost_ledger.user_id</code> nullable with <code>SET NULL</code> . Anonymize-then-cascade replaces surgical-delete. §6.8, §6.12, §10.
A3	Added <code>updated_at</code> and <code>trigger</code> to <code>session_summaries</code> and <code>retrieval_checks</code> , both regenerable per their own descriptions but missing the timestamp in v1.0. §6.11.
A4	Moved the session-cost trigger from <code>session_turns</code> (which has no cost column) to <code>agent_traces</code> . §6.6.
A5	Added missing FK indexes on <code>content_artifacts.reviewed_by</code> , <code>.superseded_by</code> ; <code>rubric_criteria.reviewed_by</code> ; <code>learner_subjects.subject_id</code> , <code>.current_focus_concept_id</code> ;

Finding Change

learning_sessions.focus_concept_id; session_turns.artifact_id;
journal_entries.first_session_id; assessment_attempts.session_id,
.scope_concept_id; review_cards.concept_id;
portfolio_items.parent_item_id. §6.4, §6.5, §6.6, §6.7, §6.8, §6.9, §6.10, §7.

Design corrections (non-blocking but material).

Area Change

Cost roll-up	Widened cost_ledger to capture five cost sources (agent_traces, content_artifacts, ingestion_jobs, session_summaries, assessment_attempts.grading_cost_usd), not one. Split cache-write tokens by TTL (5m vs 1h) because pricing differs. cost_usd becomes a generated column summing the five categories. §6.12, §8.
Graph versioning	Downgraded the promise in §6.2. The schema detects drift and offers a voluntary migration; it does not deliver row-level version immutability. This is honest; the shortcut is scheduled to be revisited before Studium reaches ~30 learners on an actively-edited subject. §6.2.
Test tiering	Reframed §14 around an offline-first tier that runs against SQLAlchemy metadata and Alembic AST rather than a live database. The tier that requires a running server is bounded and named. §14.

Editorial.

- Updated the header to reflect built-and-ratified status.
- Corrected the §7 language about FK indexing to describe an invariant the schema now actually meets.
- Added notes throughout marking the source finding for each v1.1 correction, so the diff to v1.0 is legible in place.

v1.0 — 14 August 2026. Initial specification. Marked build-ready. Contained the twenty-nine defects the subsystem-1 build subsequently found. Retained in the repository at spec/01-data-layer-v1.0.md for reference; superseded by this document for all forward work.

End of specification

This document defines the persistence layer for Studium in full. Every table, column, constraint, index, and access pattern the system will use is here or explicitly forwarded to a named subsequent spec. A senior engineer can build the database, run migrations, and open a psql shell against it using nothing but this document and standard Postgres/Alembic knowledge.

Next in sequence: **Subsystem 2 — Agent Runtime and Orchestration**, which specifies how the agent group is instantiated, how they exchange context through the tables defined

here, and what each agent's prompt structure and tool set looks like. It is written against v1.1.